SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

C PROGRAMMING ASSIGNMENT

Student Submission Template

Student Name	P H Devesh
Register Number	1911260236
Course Code & Name	CSA0209 & C Programming
Slot	B
Assignment / Task Title	C04 – Advanced Data Structures and Design Engineering
GitHub Repository URL	https://github.com/DevPH/C-CSA0209
Submission Date	_____

Instructions to Students

1. Complete every section for each assigned task.
2. Write your own explanation, algorithm/pseudocode, and program.
3. Run the program and insert a clear screenshot of the output.
4. Include test cases and a short analysis where applicable.
5. If code is maintained on GitHub, provide the correct repository link.

C04 – Advanced Data Structures and Design Engineering

Q1. Task 1 – Data Design Review: Patient Register (6 Marks)

1.1 Diagnosis of deficiencies

• Parallel arrays store one patient across several separate arrays. An insertion, deletion or sort must update every array identically; a missed movement can associate one patient's name with another patient's blood pressure.

• char phone[100][10] has room for only 9 digits plus the null terminator. A 10-digit phone number needs at least char[11]; otherwise strcpy can write beyond the array and corrupt memory.

• A single float cannot reliably represent both systolic and diastolic pressure such as 120/80. The two readings must be separate fields.

• A date stored as eight text characters is hard to validate, compare and format. A date structure with day, month and year is clearer and supports date calculations.

• There is no check that count is below 100. The next insertion can write outside every array.

• Global arrays and count create shared mutable state, make functions dependent on hidden data and complicate testing and reuse.

• Sorting by age requires coordinated swaps across all arrays, increasing the chance of mismatched patient records.

1.2 Repaired structure design

```
#include <stdio.h>
#include <string.h>

#define MAX_PATIENTS 100

typedef struct { int day, month, year; } Date;
typedef struct { int systolic, diastolic; } BloodPressure;
typedef struct {
    char name[20];
    int age;
    char phone[11];       /* 10 digits + '\0' */
    BloodPressure bp;
    Date visitDate;
} Patient;

int addPatient(Patient patients[], int *count, const Patient *p)
{
    if (*count >= MAX_PATIENTS) return 0;
    patients[*count] = *p;
    ++(*count);
    return 1;
}

int main(void)
{
    Patient patients[MAX_PATIENTS];
    int count = 0;
    Patient p = {"Arun", 20, "9876543210", {120, 80}, {9, 11, 2026}};
    if (addPatient(patients, &count, &p))
        printf("Added: %s, age %d, BP %d/%d\n",
               patients[0].name, patients[0].age,
               patients[0].bp.systolic, patients[0].bp.diastolic);
    else puts("Register is full");
    return 0;
}
```

1.3 Design defence (under 100 words)

An array of Patient structures keeps each person's fields together. A sort swaps one complete record, so names, age, phone, pressure and date cannot become separated. Functions can receive one Patient pointer instead of many unrelated arrays and indices. New fields can be added to the structure without creating another parallel array. The design is therefore easier to pass, sort, validate, test and extend.

Alternative concept and complexity

Parallel arrays can be preferable when processing one field across many records (for example, vectorised age analysis) or when a data-oriented layout improves cache use. Both designs use O(n) storage. A comparison sort takes O(n log n) comparisons on average; selection/bubble-style sorting may take O(n^2). Swapping parallel arrays moves each field separately, while swapping structures moves sizeof(Patient) bytes per swap. Padding may increase sizeof(Patient), but the structure layout simplifies correctness.

Q2. Task 2 – From Requirement to Nested Structures: Courier Parcel Tracking (8 Marks)

2.1 Entities and fields

Parcel: ID, weight, booking date, sender, receiver, service type and charge. Address: person name, city and PIN code. Date: day, month and year. A Parcel contains two Address values and one Date value; this is nested composition.

2.2 Structure definitions and array

```
#include <stdio.h>
#include <string.h>

typedef struct { int day, month, year; } Date;
typedef struct {
    char name[30], city[20];
    int pincode;
} Address;
typedef enum { NORMAL, EXPRESS } ServiceType;
typedef struct {
    int id;
    float weight, charge;
    Address sender, receiver;
    Date booked;
    ServiceType service;
} Parcel;

Parcel bookings[100];
int bookingCount = 0;
```

2.3 Pseudocode

```
SORT_BY_WEIGHT(parcels, n):
    for i = 0 to n-2:
        min = i
        for j = i+1 to n-1:
            if parcels[j].weight < parcels[min].weight:
                min = j
        if min != i:
            swap entire parcels[i] and parcels[min]

LIST_RECEIVER_CITY(parcels, n, target):
    for i = 0 to n-1:
        if parcels[i].receiver.city equals target:
            display parcel details
```

2.4 Access paths

Array with dot operator: **bookings[2].receiver.pincode**
Pointer with arrow operator: **p->receiver.pincode** (where p points to the third parcel).

Completed C program (TODO 1 and TODO 2)

```
#include <stdio.h>
struct date { int day, month, year; };
struct address { char city[20]; int pincode; };
struct parcel {
    int id; float weight;
    struct address to;
    struct date booked;
};

void print_parcel(const struct parcel *p)
{
    printf("%d %.1f kg %s-%d %02d/%02d/%04d\n",
           p->id, p->weight, p->to.city, p->to.pincode,
           p->booked.day, p->booked.month, p->booked.year);
}
void sort_by_weight(struct parcel p[], int n)
```

```
{
    for (int i=0; i<n-1; i++) {
        int min=i;
        for (int j=i+1; j<n; j++)
            if (p[j].weight < p[min].weight) min=j;
        if (min != i) {
            struct parcel temp=p[i]; p[i]=p[min]; p[min]=temp;
        }
    }
}
int main(void)
{
    struct parcel list[3] = {
      {501,4.5f,{"Madurai",625001},{12,10,2026}},
      {502,1.2f,{"Chennai",600077},{12,10,2026}},
      {503,2.8f,{"Coimbatore",641001},{13,10,2026}}
    };
    sort_by_weight(list,3);
    for(int i=0;i<3;i++) print_parcel(&list[i]);
    return 0;
}
```

Expected output

```
502 1.2 kg Chennai-600077 12/10/2026
503 2.8 kg Coimbatore-641001 13/10/2026
501 4.5 kg Madurai-625001 12/10/2026
```

Each selection-sort swap copies three structure-sized objects (temporary plus two assignments), or approximately 3 × sizeof(struct parcel) bytes of structure data movement. A flat structure places city, PIN and date fields directly in Parcel; access becomes p->city and p->day, but the Address type cannot be reused as neatly. Nested and flat layouts may have the same total field storage, though padding and alignment determine the actual sizeof; verify with sizeof on the target compiler.

Q3. Task 3 – Unions and Memory Layout: Weather Sensor Packets (9 Marks)

3.1 Structure versus tagged union

A structure containing float, int and char[8] stores space for all three alternatives even though only one value is valid per packet. A tagged union stores an enum tag plus one shared memory area. Since every packet carries exactly one kind of reading and memory is limited, the tagged union is the appropriate design.

3.2 Size prediction

On the stated typical 64-bit GCC platform, enum is commonly 4 bytes. The union size is the largest member rounded for its alignment: char[8] is 8 bytes and float/int are 4 bytes, so union size is 8. The tag (4) plus union (8) gives 12 bytes; alignment is already satisfied. A structure with all three fields needs approximately 4 + 4 + 8 = 16 bytes (subject to ABI padding). Always confirm with sizeof because C does not mandate these exact sizes.

3.3 Completed tagged-union program

```
#include <stdio.h>
enum kind { TEMPERATURE, HUMIDITY, STATUS };
struct reading {
    enum kind type;
    union {
        float celsius;
        int percent;
        char status[8];
    } value;
};
void show(const struct reading *r)
{
    switch (r->type) {
    case TEMPERATURE: printf("Temp %.1f C\n", r->value.celsius); break;
    case HUMIDITY:    printf("Humidity %d %%\n", r->value.percent); break;
    case STATUS:      printf("Status %s\n", r->value.status); break;
    }
}
int main(void)
{
    struct reading log[3] = {
        {TEMPERATURE, {.celsius=31.5f}},
        {HUMIDITY, {.percent=68}},
```

```
        {STATUS, {.status="OK"}}
    };
    printf("sizeof union = %zu, sizeof struct reading = %zu\n",
           sizeof log[0].value, sizeof log[0]);
    for(int i=0;i<3;i++) show(&log[i]);
    return 0;
}
```

Expected output on the stated platform

```
sizeof union = 8, sizeof struct reading = 12
Temp 31.5 C
Humidity 68 %
Status OK
```

3.4 Misuse experiment

After writing celsius and then reading percent, the program reads the same union bytes using a different member type. The result is not a meaningful humidity value and is implementation-dependent; it must not be used as a valid reading. The tag must always match the last member written.

Alternative designs

Three separate arrays can avoid storing a tag per item, but the program must maintain type-specific indices and map a packet to the correct array. Access is O(1) when the type and index are known. A void-pointer design typically requires dynamic allocation, a tag elsewhere, pointer storage and allocation overhead; dereferencing is O(1), but memory fragmentation and allocation costs make it less suitable for thousands of tiny packets. The tagged union gives compact contiguous storage and O(1) dispatch.

Q4. Task 4 – Structure Forensics: Bank Accounts (7 Marks)

4.1 Defects

addinterest receives struct account by value, so it changes only a local copy. The original list element remains unchanged. In main, p is a pointer, so p.number and p.balance are invalid; use p->number and p->balance or list[0].number.

4.2 Address evidence

Print (void *)&a inside the by-value function and (void *)&list[i] in main. The addresses differ, proving the function parameter is a separate copy.

4.3 Repaired program using pointer and arrow operator

```
#include <stdio.h>
struct account { int number; float balance; };
void addinterest(struct account *a, float rate)
{
    a->balance += a->balance * rate / 100.0f;
}
int main(void)
{
    struct account list[3]={{101,5000},{102,12000},{103,800}};
    struct account *p=list;
    for(int i=0;i<3;i++) addinterest(&list[i],4.0f);
    for(int i=0;i<3;i++)
        printf("%d %.2f\n", p[i].number, p[i].balance);
    return 0;
}
```

Expected output

```
101 5200.00
102 12480.00
103 832.00
```

4.4 Passing guidelines

Pass by value for small structures when an independent copy is desired. Pass by pointer when the function must modify the original or copying a large structure is wasteful. Pass by const pointer when the function only reads the structure; this avoids copying while preventing modification through that pointer.

Alternative complexity

Returning a modified structure by value is correct: struct account addinterest(struct account a, float rate) { a.balance += a.balance*rate/100; return a; } and call list[i] = addinterest(list[i],4). Both approaches take O(1) time per account. Value/return may copy sizeof(struct account) bytes into the parameter and back; a pointer passes one address and uses a small parameter slot. Actual compilers may optimise copies, but pointer passing is usually preferable for large structures.

Q5. Task 5 – Storage Classes: The Same Counter Four Ways (7 Marks)

5.1 Implementations and observed tokens

Automatic local variable: it is created on each call and reset to 1000, so three calls return 1001, 1001, 1001. Static local variable: retains its value between calls, producing 1001, 1002, 1003. A global variable shared between source files is defined once (e.g. int token=1000 in one .c file) and declared extern int token in the other; each call increments the same object. A register loop variable is only a suggestion to the compiler to keep the loop counter in a CPU register; it is not suitable as the persistent token counter.

5.1 Completed code: static and automatic

```
#include <stdio.h>
int next_token(void)
{
    static int token = 1000;
    return ++token;
}
int auto_token(void)
{
    int token = 1000;
    return ++token;
}
int main(void)
{
    int s1=next_token(), s2=next_token(), s3=next_token();
    int a1=auto_token(), a2=auto_token(), a3=auto_token();
    printf("static: %d %d %d\n",s1,s2,s3);
    printf("auto : %d %d %d\n",a1,a2,a3);
    return 0;
}
```

Expected output

```
static: 1001 1002 1003
auto : 1001 1001 1001
```

Global counter shared across two files

```
/* counter.c */
int shared_token = 1000;

/* token_api.c */
extern int shared_token;
int next_global_token(void) { return ++shared_token; }
```

Register loop variable example

```
for (register int i=0; i<3; ++i)
    printf("Token %d\n", 1001+i);
```

5.2 Two independent counters

Use two static variables (veg and nonveg) inside separate functions, two separate global variables with extern declarations, or two caller-owned integers passed by pointer. The automatic version still resets every call and cannot independently maintain either sequence without changing its lifetime strategy. The register keyword does not provide persistent storage.

5.3 Comparison

Automatic: block scope, call lifetime, indeterminate initial value unless initialised, recreated each call. Static local: block scope, entire program lifetime, zero-initialised by default, retains state. Global: file scope by default (extern exposes the same object elsewhere), entire program lifetime, zero-initialised, any code with access may modify it. Register: block scope and automatic lifetime; register is a hint and the compiler may ignore it. The language does not allow taking the address of an object declared register because it may reside only in a CPU register.

Evaluation order

Do not call next_token() three times as arguments of one printf and assume left-to-right evaluation. C does not specify the order of function-argument evaluation, so the values may appear in a different order. Call the function three times in separate statements, store the results, and then print them.

Caller-owned counter alternative

```
int next_token(int *counter)
{
    return ++(*counter);
}
int main(void)
{
    int veg=1000, nonveg=1000;
    printf("%d %d %d\n", next_token(&veg),
           next_token(&nonveg), next_token(&veg));
}
```

Each call is O(1). The counter occupies one int in the caller's storage and lives as long as that variable. Two independent counters are straightforward: pass different addresses. This design avoids hidden global state and is easiest to test.

Q6. Task 6 – Design Challenge: College Canteen Inventory and Billing (13 Marks)

Stage 1 – Requirements and rules

Entities: Item, SaleLine, Bill, Date and Time. Every item has an ID, name, price and stock quantity. Packaged items also have an expiry date; cooked items have a preparation time; combo meals contain up to three component item IDs and quantities. A sale must reject quantities greater than available stock, reduce stock only after validation, calculate line revenue, and add it to the daily summary. Packaged items expiring within the next two days must trigger a warning. The daily summary must be sorted by revenue.

Stage 2 – Data model

```
#include <stdio.h>
#define MAX_ITEMS 100
#define MAX_LINES 50
typedef struct { int day, month, year; } Date;
typedef struct { int hour, minute; } Time;
typedef enum { PACKAGED, COOKED, COMBO } ItemType;
typedef struct { int itemId, quantity; } Component;
typedef struct {
    int id;
    char name[40];
    float price;
    int stock;
    ItemType type;
    union {
        Date expiry;                /* PACKAGED */
        Time prepared;              /* COOKED */
        struct { Component c[3]; int count; } combo;
    } detail;
    float revenue;
} Item;
typedef struct { int itemId, quantity; float amount; } SaleLine;
typedef struct { SaleLine lines[MAX_LINES]; int count; float total; } Bill;
```

Stage 3 – Critical design decisions

Use an array of Item structures for a bounded canteen catalogue. The tagged union stores only the extra data valid for the item type. Before a sale, locate the item, verify quantity is positive and stock is sufficient, then update stock and revenue as one logical operation. Use a date structure and a date-comparison helper to identify expiry dates from today through today + 2 days. No global variable is necessary for stock or bills; pass pointers to the inventory and bill. A static/global counter is justified only if a unique bill number must persist across function calls and is not already stored in a database/file.

Stage 4 – Function prototypes and pseudocode

```
int find_item(const Item items[], int n, int id);
int record_sale(Item items[], int n, Bill *bill,
                int itemId, int quantity);
void warn_expiring(const Item items[], int n, Date today);
void sort_summary(Item items[], int n);
```

```
/* RECORD SALE:
   locate item by ID
   if missing, quantity <= 0, or quantity > stock: reject
   amount = price * quantity
   decrease stock by quantity
   increase item revenue by amount
   append SaleLine to bill and increase bill total
   report success */

/* EXPIRY WARNING:
   for each item:
     if type is PACKAGED and expiry is between today and today+2:
        print item and expiry date */

/* SORT SUMMARY:
   selection-sort item records by revenue descending */
```

Stage 5 – Working demonstration program

```
#include <stdio.h>
#include <string.h>
#define N 4
typedef struct { int day,month,year; } Date;
typedef enum { PACKAGED, COOKED, COMBO } Type;
typedef struct { int id,qty; } Part;
typedef struct {
 int id; char name[24]; float price; int stock; Type type;
 union { Date expiry; int preparedMinute;
         struct { Part p[3]; int count; } combo; } extra;
 float revenue;
} Item;
int find_item(Item a[], int n, int id) {
 for(int i=0;i<n;i++) if(a[i].id==id) return i;
 return -1;
}
int sale(Item a[], int n, int id, int qty) {
 int k=find_item(a,n,id);
 if(k<0 || qty<=0 || qty>a[k].stock) return 0;
 a[k].stock-=qty; a[k].revenue+=a[k].price*qty;
 printf("Sold %d x %s = %.2f\n",qty,a[k].name,a[k].price*qty);
 return 1;
}
void expiry_warning(Item a[], int n) {
 for(int i=0;i<n;i++) if(a[i].type==PACKAGED &&
    a[i].extra.expiry.year==2026 &&
    a[i].extra.expiry.month==11 &&
    a[i].extra.expiry.day<=11)
   printf("Expiry warning: %s\n",a[i].name);
}
void sort_revenue(Item a[], int n) {
 for(int i=0;i<n-1;i++) {
   int best=i;
   for(int j=i+1;j<n;j++)
     if(a[j].revenue>a[best].revenue) best=j;
   if(best!=i) { Item t=a[i]; a[i]=a[best]; a[best]=t; }
 }
}
int main(void) {
 Item a[N]={
  {1,"Milk",30,10,PACKAGED,{{9,11,2026}},0},
  {2,"Sandwich",50,8,COOKED,{.preparedMinute=600},0},
  {3,"Juice",25,3,PACKAGED,{{12,11,2026}},0},
  {4,"Combo Meal",90,5,COMBO,{.combo={{ {2,1},{3,1} },2},},0}
 };
 sale(a,N,1,2);
 if(!sale(a,N,3,5)) puts("Rejected: insufficient stock");
 expiry_warning(a,N);
 sort_revenue(a,N);
 puts("Daily revenue summary (descending):");
 for(int i=0;i<N;i++)
   printf("%s : %.2f (stock %d)\n",a[i].name,a[i].revenue,a[i].stock);
 return 0;
}
```

Illustrative output

```
Sold 2 x Milk = 60.00
Rejected: insufficient stock
Expiry warning: Milk
Daily revenue summary (descending):
Milk : 60.00 (stock 8)
Sandwich : 0.00 (stock 8)
Juice : 0.00 (stock 3)
Combo Meal : 0.00 (stock 5)
```

Note on expiry validation

The compact demonstration uses a fixed sample date check for the sample data. A production solution should implement a calendar-aware date difference function, including month lengths and leap years, and compare expiry against today and the date two days later. Combo sale processing should recursively or iteratively validate all component stock before decrementing any component, so a partial combo cannot be sold.

Alternative: linked list versus array

A linked list can add a new item at the head in O(1), remove a known node in O(1) when its predecessor is known, and search by ID in O(n). Removing an item by ID still requires O(n) search. An array supports O(1) indexed access and good cache locality; appending is O(1) when capacity exists, but inserting/removing in the middle requires O(n) shifts. A linked list uses extra memory for one next pointer per node and incurs allocation overhead and poorer locality. For a small, fixed-size canteen catalogue, an array is usually simpler and faster; a linked list is useful when frequent insertions/deletions and variable size dominate.

Conclusion

This assignment demonstrates how C structures, nested structures, unions, pointers and storage classes support safer data modelling and practical program design. Selecting an appropriate representation reduces memory waste, prevents data mismatches and makes operations easier to test and maintain.